QUESTION-17:

Illustrate the deadlock avoidance concept by simulating Banker's algorithm using C.

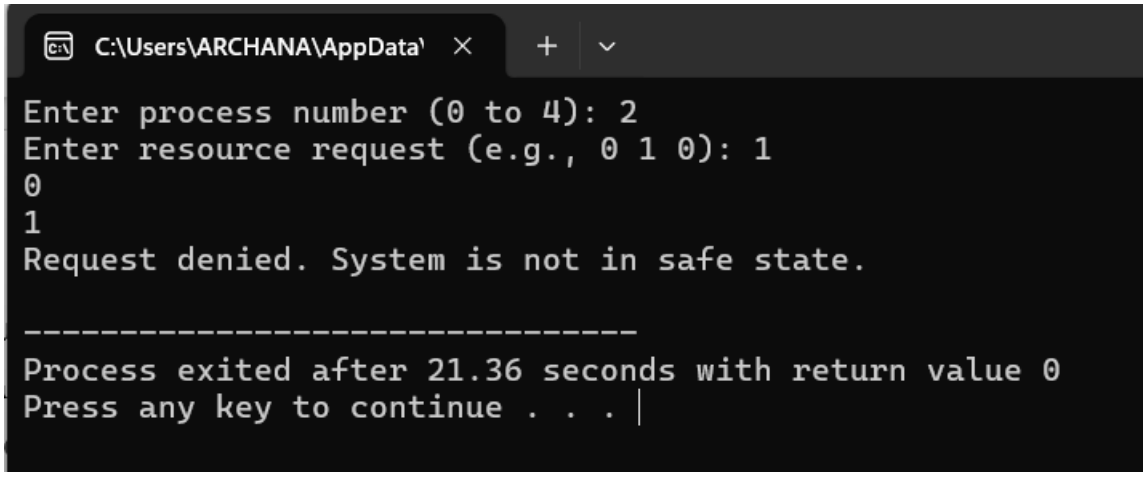
PROGRAM:

```
#include <stdio.h>
#define MAX_PROCESSES 5
#define MAX_RESOURCES 3
int is_safe();
int available[MAX_RESOURCES] = {3, 3, 2}; // Available instances of each resource
int maximum[MAX_PROCESSES][MAX_RESOURCES] = {{7, 5, 3},
{3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
int allocation[MAX_PROCESSES][MAX_RESOURCES] = {{0, 1, 0},
{2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};
int request_resources(int process_num, int request[]) {
// Check if request can be granted
for (int i = 0; i < MAX_RESOURCES; i++) {
if (request[i] > available[i] || request[i] > maximum[process_num][i] - allocation[process_num][i])
return 0; // Request cannot be granted
}
// Try allocating resources temporarily
for (int i = 0; i < MAX_RESOURCES; i++) {
available[i] -= request[i];
allocation[process_num][i] += request[i];
maximum[process_num][i] -= request[i];
}
// Check if system is in safe state after allocation
if (is_safe()) {
return 1; // Request is granted
} else {
// Roll back changes if not safe
for (int i = 0; i < MAX_RESOURCES; i++) {
```

```
available[i] += request[i];
allocation[process_num][i] -= request[i];
maximum[process_num][i] += request[i];
}
return 0; // Request is denied
}
}
int is_safe() {
int work[MAX_RESOURCES];
int finish[MAX_PROCESSES] = {0};
// Initialize work array
for (int i = 0; i < MAX_RESOURCES; i++) {
work[i] = available[i];
}
// Check if processes can finish
int count = 0;
while (count < MAX_PROCESSES) {
int found = 0;
for (int i = 0; i < MAX_PROCESSES; i++) {
if (finish[i] == 0) {
int j;
for (j = 0; j < MAX_RESOURCES; j++) {
if (maximum[i][j] - allocation[i][j] > work[j])
break;
}
if (j == MAX_RESOURCES) {
// Process can finish, update work and mark as finished
for (int k = 0; k < MAX_RESOURCES; k++) {
work[k] += allocation[i][k];
}
```

```
finish[i] = 1;
found = 1;
count++;
}
}
}
if (found == 0) {
return 0; // No process can finish, not safe state
}
}
return 1; // All processes can finish, safe state
}
int main() {
int process_num, request[MAX_RESOURCES];
printf("Enter process number (0 to 4): ");
scanf("%d", &process_num);
printf("Enter resource request (e.g., 0 1 0): ");
for (int i = 0; i < MAX_RESOURCES; i++) {
scanf("%d", &request[i]);
}
if (request_resources(process_num, request)) {
printf("Request granted.\n");
} else {
printf("Request denied. System is not in safe state.\n");
}
return 0;
}
```

OUTPUT:



QUESTION-18:

Construct a C program to simulate producer consumer problem using semaphores.

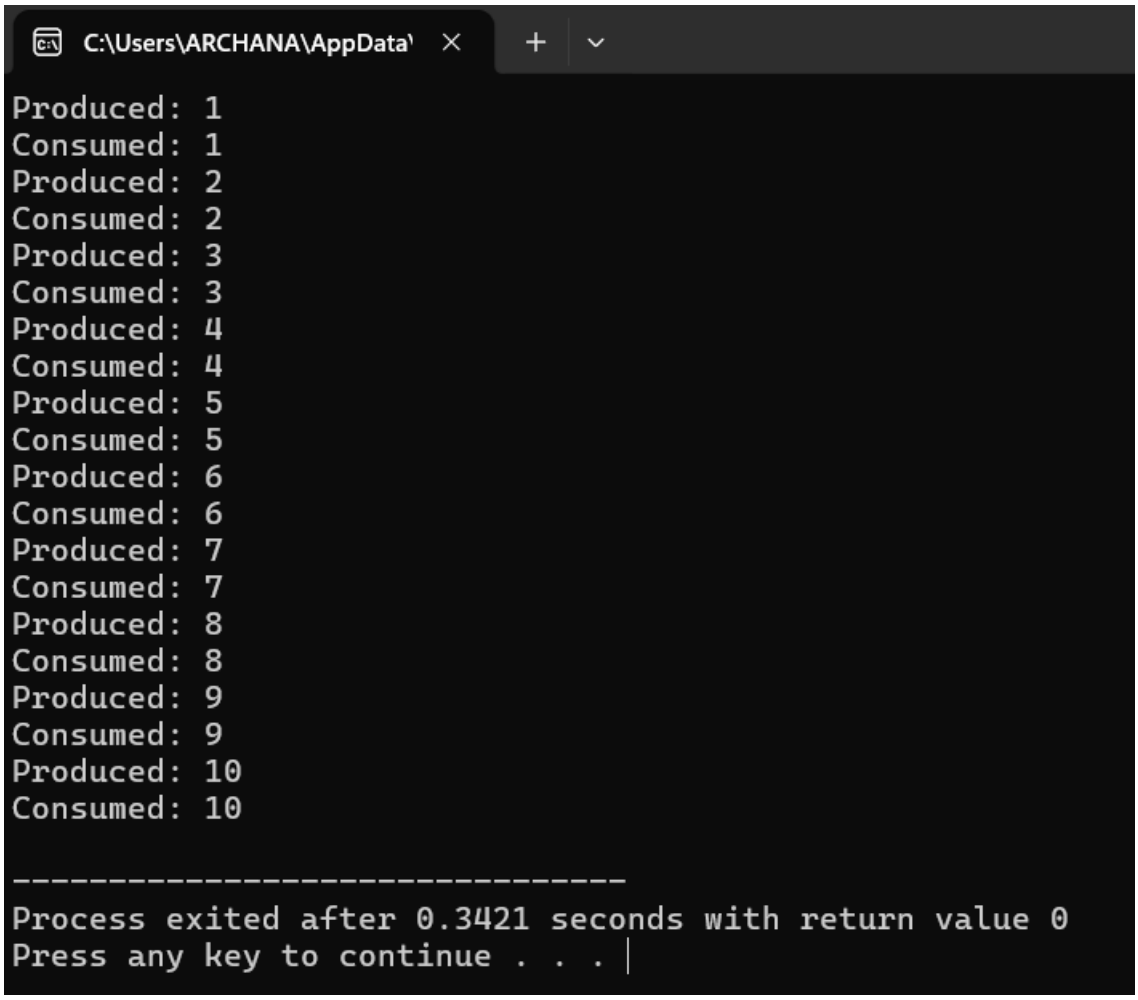
PROGRAM:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <Windows.h>
#define BUFFER_SIZE 5
#define MAX_ITEMS 10 // Maximum number of items to be produced/consumed
int buffer[BUFFER_SIZE];
sem_t empty, full;
int produced_items = 0, consumed_items = 0;
void* producer(void* arg) {
    while (produced_items < MAX_ITEMS) {
        sem_wait(&empty);
        // Critical section: add item to buffer
        for (int i = 0; i < BUFFER_SIZE; ++i) {
            if (buffer[i] == 0) {
                buffer[i] = produced_items + 1;
                printf("Produced: %d\n", buffer[i]);
```

```
                produced_items++;
                break;
            }
        }
        sem_post(&full);
        Sleep(1); // Sleep for a while
    }
    return NULL;
}
void* consumer(void* arg) {
    while (consumed_items < MAX_ITEMS) {
        sem_wait(&full);
        // Critical section: remove item from buffer
        for (int i = 0; i < BUFFER_SIZE; ++i) {
            if (buffer[i] != 0) {
                printf("Consumed: %d\n", buffer[i]);
                buffer[i] = 0;
                consumed_items++;
                break;
            }
        }
        sem_post(&empty);
        Sleep(2); // Sleep for a while
    }
    return NULL;
}
int main() {
    pthread_t producer_thread, consumer_thread;
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
```

```
    // Create producer and consumer threads
    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);
    // Wait for threads to finish
    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);
    // Destroy semaphores
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

OUTPUT:



QUESTION-19:

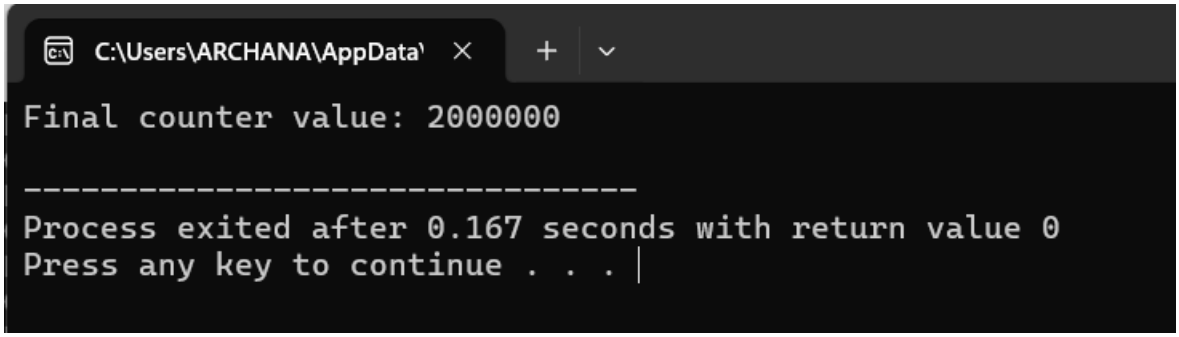
Design a C program to implement process synchronization using mutex locks.

PROGRAM:

```
#include <stdio.h>
#include <pthread.h>
// Shared variables
int counter = 0;
pthread_mutex_t mutex;
// Function to be executed by threads
void *threadFunction(void *arg) {
    int i;
    for (i = 0; i < 1000000; ++i) {
        pthread_mutex_lock(&mutex);
        counter++;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
int main() {
    pthread_mutex_init(&mutex, NULL);
    pthread_t thread1, thread2;
    pthread_create(&thread1, NULL, threadFunction, NULL);
    pthread_create(&thread2, NULL, threadFunction, NULL);
    // Wait for the threads to finish
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    // Destroy the mutex
    pthread_mutex_destroy(&mutex);
    // Print the final value of the counter
    printf("Final counter value: %d\n", counter);
```

```
    return 0;
}
```

OUTPUT:



QUESTION-20:

Construct a C program to simulate Reader-Writer problem using semaphores.

PROGRAM:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
sem_t mutex, writeBlock;
int data = 0, readersCount = 0;
void *reader(void *arg) {
    int i=0;
    while (i<10) {
        sem_wait(&mutex);
        readersCount++;
        if (readersCount == 1) {
            sem_wait(&writeBlock);
        }
        sem_post(&mutex);
        // Reading operation
```

```
        printf("Reader reads data: %d\n", data);

        sem_wait(&mutex);
        readersCount--;
        if (readersCount == 0) {
            sem_post(&writeBlock);
        }
        sem_post(&mutex);
        i++;
    }
}
void *writer(void *arg) {
    int i=0;
    while (i<10) {
        sem_wait(&writeBlock);

        // Writing operation
        data++;
        printf("Writer writes data: %d\n", data);

        sem_post(&writeBlock);
        i++;
    }
}
int main() {
    pthread_t readers, writers;
    sem_init(&mutex, 0, 1);
    sem_init(&writeBlock, 0, 1);
    pthread_create(&readers, NULL, reader, NULL);
    pthread_create(&writers, NULL, writer, NULL);
```

```
    pthread_join(readers, NULL);
    pthread_join(writers, NULL);
    sem_destroy(&mutex);
    sem_destroy(&writeBlock);
    return 0;
}
```

OUTPUT:

